

بِسْمِ اللَّهِ الرَّحْمَنِ الرَّحِيمِ

Page Replacement Algorithms

Presented By;

Huzaifa Shahbaz(21-CE-009)

Muhammad Ibthesam (21-ce-020)

Yawar Ali Malik (21-CE-021)

OUTLINES

- Abstract
- Introduction
- Problem Statement
- Objective
- Expected Outcome
- Components Used
- Project Diagram
- First In, First Out
- First In, First Out diagram
- CODE OF First In, First Out diagram
- Least recently used

OUTLINES(continued)

- Least Recently used diagram
- CODE OF Least Recently used
- Least Frequently Used
- Least Frequently Used diagram
- CODE OF Least Frequently Used
- Optimal Page Replacement
- Optimal Page Replacement Diagram
- CODE OF Optimal Page Replacement
- CODE OF Page Replacement Algorithms
- Results of Page Replacement Algorithms
- Conclusion

Abstract

- Page replacement algorithms are critical for managing a computer's memory by deciding which pages to swap out when new pages need to be loaded.
- Common algorithms include FIFO (First-In-First-Out), LRU (Least Recently Used), and Optimal Page Replacement, each with distinct strategies for determining which page to replace.
- These algorithms balance efficiency and complexity to maintain effective memory utilization.
- Research in this area continues to evolve, enhancing system performance and resource management.

Introduction

- Paging is a type of memory management scheme which the computer uses to store and receive data from the secondary storage and uses that in the main memory.
- This scheme allows the physical address space of a process to be non-contiguous.
- **Example**
- There is a process of size: 4
- Bytes The size of a single page: 2
- Bytes Number of pages/Process: $4/2 = 2$
- To follow the concept of paging the MMU makes a page table.
- Every process has its own page table.
- The number of pages in a page table is the same as the number of pages in a process.

Introduction(continued)

- **Example**
- The process 1 above, will have two entries in its page table.
- The page table however contains the “Frame No” which is the address in main memory where the data required is stored.

Problem Statement

- Developing a paging memory management system is essential for efficient data handling in computers.
- This project aims to design an optimized paging mechanism where the Memory Management Unit (MMU) maintains page tables for each process, correlating virtual addresses to physical memory locations.
- Challenges include optimizing page table size, minimizing access latency, and ensuring smooth data transfer between secondary and primary storage, providing an efficient solution for modern computing.

Objective

- The objective of this project is to implement a paging memory management system to efficiently manage data in computer systems.
- This involves designing an optimized paging mechanism where the Memory Management Unit (MMU) maintains page tables for each process.
- The goal is to address challenges such as optimizing page table size, minimizing access latency, and ensuring seamless data transfer between secondary and primary storage, ultimately enhancing overall system performance and resource utilization.

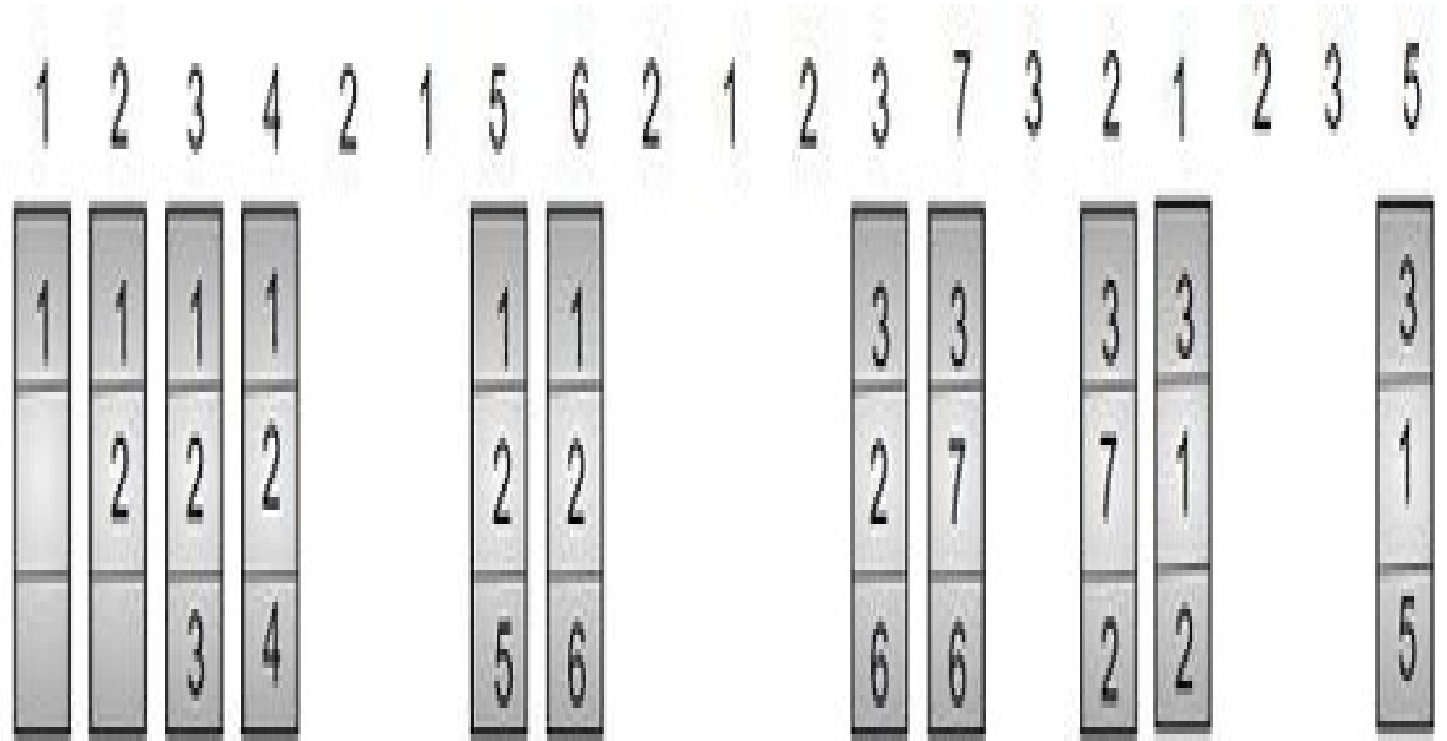
Expected Outcome

- The expected outcome of implementing an effective page replacement algorithm is a reduction in the number of page faults, leading to improved system performance and faster execution times.
- Enhanced memory utilization ensures that the most frequently accessed data is readily available, minimizing delays.
- This contributes to an overall smoother and more efficient operation of applications.
- Additionally, a well-chosen algorithm can extend the lifespan of hardware by reducing unnecessary wear from excessive data swapping.

Components Used

- The Components used of page replacement algorithm following are
- Ubnutu
- Sublime Text editor
- GCC Compiler
- Terminal

Project Diagram Page Replacement Algorithm Diagram



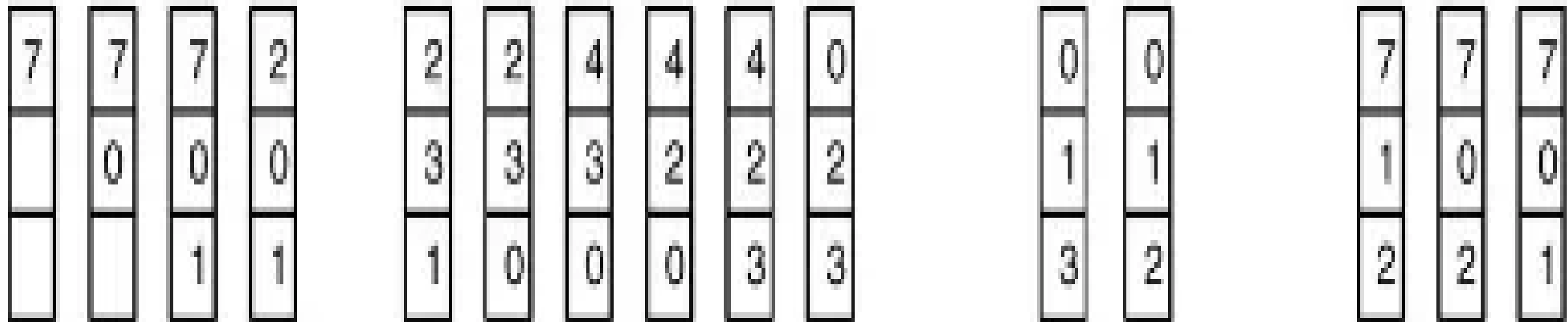
First In-First Out

- This algorithm replaces the oldest page in memory when a page fault occurs.
- It keeps track of all pages in memory in a queue and when a page fault occurs, the page at the front of the queue is removed and a new page is added to the back of the queue.

First In-First Out

reference string

7 0 1 2 0 3 0 4 2 3 0 3 2 1 2 0 1 7 0 1



page frames

Code OF First In-First Out

- `void fifo(int pages[], int n, int capacity)`
- `{`
- `int frame[capacity], front = 0, page_faults = 0;`
- `for (int i = 0; i < capacity; i++)`
- `frame[i] = -1;`
- `for (int i = 0; i < n; i++)`
- `{`
- `int found = 0;`
- `for (int j = 0; j < capacity; j++)`
- `{`

Code OF First In-First Out(CONTINUED)

- if (frame[j] == pages[i])
- {
- found = 1;
- break;
- }
- }
- if (!found)
- {
- frame[front] = pages[i];

Code OF First In-First Out(CONTINUED)

- `front = (front + 1) % capacity;`
- `page_faults++;`
- `}`
- `}`
- `printf("FIFO Page Faults: %d\n", page_faults);`
- `}`

Least Recently Used

- The LRU page replacement algorithm replaces the least recently used page in the memory with a new page.
- The LRU page replacement algorithm is based on the assumption that among all pages in the memory, the least recently used page will not be used for a long time.

Least Recently Used Diagram

Page
reference

7,0,1,2,0,3,0,4,2,3,0,3,2,3

No. of Page frame - 4

7	0	1	2	0	3	0	4	2	3	0	3	2	3
			2	2	2	2	2	2	2	2	2	2	2
		1	1	1	1	1	4	4	4	4	4	4	4
	0	0	0	0	0	0	0	0	0	0	0	0	0
7	7	7	7	7	3	3	3	3	3	3	3	3	3
Miss	Miss	Miss	Miss	Hit	Miss	Hit	Miss	Hit	Hit	Hit	Hit	Hit	Hit

Total Page Fault = 6

Code of Least Recently Used

- `void lru(int pages[], int n, int capacity)`
- `{`
- `int frame[capacity], time[capacity], page_faults = 0, counter = 0;`
- `for (int i = 0; i < capacity; i++)`
- `frame[i] = -1;`
- `for (int i = 0; i < n; i++)`
- `{`
- `int found = 0, lru_index = 0;`
- `for (int j = 0; j < capacity; j++)`

Code of Least Recently Used(Continued)

- {
- if (frame[j] == pages[i])
- {
- found = 1;
- time[j] = ++counter;
- break;
- }
- }
- if (!found)
- {

Code of Least Recently Used(Continued)

- `int min_time = counter + 1;`
- `for (int j = 0; j < capacity; j++)`
- `{`
- `if (time[j] < min_time)`
- `{`
- `min_time = time[j];`
- `lru_index = j;`
- `}`
- `}`

Code of Least Recently Used(Continued)

- `frame[lru_index] = pages[i];`
- `time[lru_index] = ++counter;`
- `page_faults++;`
- `}`
- `}`
- `printf("LRU Page Faults: %d\n", page_faults);`
- `}`

Least Frequently Used

- It removes the page that has not been utilized in the memory for the longest period of time.
- It replaces the least frequently used pages. It keeps track of page usage in the memory over a short time period.

Least Frequently Used Diagram

String	7	0	2	4	3	1	4	7	2	0	4	3	0	3	2	7
Frame 3			2	2	2	1	1	1	2	2	2	3	3	3	3	3
Frame 2		0	0	0	3	3	3	7	7	0	0	0	0	0	2	7
Frame 1	7	7	7	4	4	4	4	4	4	4	4	4	4	4	4	4
Miss/Hit	M	M	M	M	M	M	H	M	M	M	H	M	H	H	M	M

CODE OF Least Frequently Used

- `void lfu(int pages[], int n, int capacity)`
- `{`
- `int frame[capacity], count[capacity], page_faults = 0;`
- `for (int i = 0; i < capacity; i++)`
- `{`
- `frame[i] = -1;`
- `count[i] = 0;`
- `}`
- `for (int i = 0; i < n; i++)`
- `{`

CODE OF Least Frequently Used (Continued)

- `int found = 0, lfu_index = 0;`
- `for (int j = 0; j < capacity; j++)`
- `{`
- `if (frame[j] == pages[i])`
- `{`
- `found = 1;`
- `count[j]++;`
- `break;`
- `}`
- `}`

CODE OF Least Frequently Used (Continued)

- if (!found)
- {
- int min_count = n + 1;
- for (int j = 0; j < capacity; j++)
- {
- if (frame[j] == -1)
- {
- lfu_index = j;
- break;
- }
- else if (count[j] < min_count)
- {
- min_count = count[j];

CODE OF Least Frequently Used (Continued)

- lfu_index = j;
- }
- }
- frame[lfu_index] = pages[i];
- count[lfu_index] = 1;
- page_faults++;
- }
- }
- printf("LFU Page Faults: %d\n", page_faults);
- }

Optimal Page Replacement

- The optimal page replacement (OPT) algorithm is a memory management technique.
- It minimizes the number of page faults by predicting future accesses and replacing the least recently used pages.

Optimal Page Replacement Diagram

7 0 1 2 0 3 0 4 2 3

		1	1	1	3	3	3	3	3
	0	0	0	0	0	0	4	4	4
7	7	7	2	2	2	2	2	2	2

M M M M H M H M H H

CODE OF Optimal Page Replacement

- `void optimal(int pages[], int n, int capacity)`
- `{`
- `int frame[capacity], page_faults = 0;`
- `for (int i = 0; i < capacity; i++)`
- `frame[i] = -1;`
- `for (int i = 0; i < n; i++)`
- `{`
- `int found = 0, replace_index = 0;`
- `for (int j = 0; j < capacity; j++)`
- `{`

CODE OF Optimal Page Replacement (Continued)

- if (frame[j] == pages[i])
- {
- found = 1;
- break;
- }
- }
- if (!found)
- {
- int farthest = i + 1;

CODE OF Optimal Page Replacement (Continued)

- for (int j = 0; j < capacity; j++)
- {
- int k;
- for (k = i + 1; k < n; k++)
- {
- if (frame[j] == pages[k])
- break;
- }
- if (k == n)
- {
- replace_index = j;
- break;

CODE OF Optimal Page Replacement (Continued)

- for (int j = 0; j < capacity; j++)
- {
- int k;
- for (k = i + 1; k < n; k++)
- {
- if (frame[j] == pages[k])
- break;
- }
- if (k == n)
- {
- replace_index = j;
- break;

CODE OF Optimal Page Replacement (Continued)

- }
- else if ($k > \text{farthest}$)
- {
- $\text{farthest} = k;$
- $\text{replace_index} = j;$
- }
- }
- $\text{frame}[\text{replace_index}] = \text{pages}[i];$
- $\text{page_faults}++;$
- }
- }
- $\text{printf}(\text{"Optimal Page Faults: \%d\\n"}, \text{page_faults});$
- }

Code of Page Replacement Algorithm

```
1  #include <stdio.h>
2  #include <string.h>
3  // Function to implement FIFO page replacement algorithm
4  void fifo(int pages[], int n, int capacity) {
5      int frame[capacity], front = 0, page_faults = 0;
6      for (int i = 0; i < capacity; i++)
7          frame[i] = -1;
8      for (int i = 0; i < n; i++) {
9          int found = 0;
10         for (int j = 0; j < capacity; j++) {
11             if (frame[j] == pages[i]) {
12                 found = 1;
13                 break;
14             }
15         }
16         if (!found) {
17             frame[front] = pages[i];
18             front = (front + 1) % capacity;
19             page_faults++;
20         }
21
22         printf("FIFO Page Faults: %d\n", page_faults);
23     }
24     // Function to implement LRU page replacement algorithm
25     void lru(int pages[], int n, int capacity) {
26         int frame[capacity], time[capacity], page_faults = 0, counter = 0;
27         for (int i = 0; i < capacity; i++)
28             frame[i] = -1;
29
30         for (int i = 0; i < n; i++) {
31             int found = 0, lru_index = 0;
32             for (int j = 0; j < capacity; j++) {
33                 if (frame[j] == pages[i]) {
34                     found = 1;
```

Code of Page Replacement Algorithm(Continued)

```
35         time[j] = ++counter;
36         break;
37     }
38 }
39 if (!found) {
40     int min_time = counter + 1;
41     for (int j = 0; j < capacity; j++) {
42         if (time[j] < min_time) {
43             min_time = time[j];
44             lru_index = j;
45         }
46     }
47     frame[lru_index] = pages[i];
48     time[lru_index] = ++counter;
49     page_faults++;
50 }
51 }
52
53 printf("LRU Page Faults: %d\n", page_faults);
54 }
55
56 // Function to implement LFU page replacement algorithm
57 void lfu(int pages[], int n, int capacity) {
58     int frame[capacity], count[capacity], page_faults = 0;
59
60     for (int i = 0; i < capacity; i++) {
61         frame[i] = -1;
62         count[i] = 0;
63     }
64
65     for (int i = 0; i < n; i++) {
66         int found = 0, lfu_index = 0;
67
68         for (int j = 0; j < capacity; j++) {
69             if (frame[j] == pages[i]) {
70                 found = 1;
71                 count[j]++;
```

Code of Page Replacement Algorithm(Continued)

```
72         break;
73     }
74 }
75
76 if (!found) {
77     int min_count = n + 1;
78     for (int j = 0; j < capacity; j++) {
79         if (frame[j] == -1) {
80             lfu_index = j;
81             break;
82         } else if (count[j] < min_count) {
83             min_count = count[j];
84             lfu_index = j;
85         }
86     }
87
88     frame[lfu_index] = pages[i];
89     count[lfu_index] = 1;
90     page_faults++;
91 }
92 }
93
94 printf("LFU Page Faults: %d\n", page_faults);
95 }
96
97 // Function to implement Optimal page replacement algorithm
98 void optimal(int pages[], int n, int capacity) {
99     int frame[capacity], page_faults = 0;
100
101     for (int i = 0; i < capacity; i++)
102         frame[i] = -1;
103
104     for (int i = 0; i < n; i++) {
105         int found = 0, replace_index = 0;
106
107         for (int j = 0; j < capacity; j++) {
108             if (frame[j] == pages[i]) {
109                 found = 1;
```

Code of Page Replacement Algorithm(Continued)

```
110         break;
111     }
112 }
113
114 if (!found) {
115     int farthest = i + 1;
116
117     for (int j = 0; j < capacity; j++) {
118         int k;
119         for (k = i + 1; k < n; k++) {
120             if (frame[j] == pages[k])
121                 break;
122         }
123         if (k == n) {
124             replace_index = j;
125             break;
126         } else if (k > farthest) {
127             farthest = k;
128             replace_index = j;
129         }
130     }
131
132     frame[replace_index] = pages[i];
133     page_faults++;
134 }
135 }
136 printf("Optimal Page Faults: %d\n", page_faults);
137 }
138 // Main function to run the page replacement algorithms based on user input
139 int main(int argc, char *argv[]) {
140     if (argc != 2) {
141         printf("Usage: %s <algorithm>\n", argv[0]);
142         printf("Algorithms: fifo, lru, lfu, optimal\n");
143         return 1;
144     }
145     int pages[] = {1, 3, 0, 3, 5, 6};
146     int n = sizeof(pages) / sizeof(pages[0]);
147     int capacity = 3;
```


Code of Page Replacement Algorithm(Continued)

```
148     if (strcmp(argv[1], "fifo") == 0) {
149         fifo(pages, n, capacity);
150     } else if (strcmp(argv[1], "lru") == 0) {
151         lru(pages, n, capacity);
152     } else if (strcmp(argv[1], "lfu") == 0) {
153         lfu(pages, n, capacity);
154     } else if (strcmp(argv[1], "optimal") == 0) {
155         optimal(pages, n, capacity);
156     } else {
157         printf("Invalid algorithm. Use fifo, lru, lfu, or optimal.\n");
158         return 1;
159     }
160     return 0;
161 }
```

Results Of page replacement algorithm

```
huzatifashahbazz@huzatifashahbazz-VirtualBox:~/huzatifa802$ subl pagereplacement.c
huzatifashahbazz@huzatifashahbazz-VirtualBox:~/huzatifa802$ gcc pagereplacement.c -o pagereplacement
huzatifashahbazz@huzatifashahbazz-VirtualBox:~/huzatifa802$ ./pagereplacement fifo
FIFO Page Faults: 5
huzatifashahbazz@huzatifashahbazz-VirtualBox:~/huzatifa802$ ./pagereplacement lru
LRU Page Faults: 5
huzatifashahbazz@huzatifashahbazz-VirtualBox:~/huzatifa802$ ./pagereplacement lfu
LFU Page Faults: 5
huzatifashahbazz@huzatifashahbazz-VirtualBox:~/huzatifa802$ ./pagereplacement optimal
Optimal Page Faults: 5
```

Conclusion

- In conclusion, the developed paging memory management system optimizes data handling in computer systems.
- By maintaining page tables for each process, minimizing access latency, and ensuring efficient data transfer, overall system performance is enhanced.
- Further refinement will continue to advance modern computing technologies.